

TỪ COMMIT ĐẾN CONTAINER

Mười chín sự cố có thật, và những nguyên tắc rút ra từ chúng

Git • GitHub CLI • Dockerfile • Ảnh Docker • CI/CD

Ghi chép từ nhật ký triển khai dự án **RapPhim WareHouse**
nguyenquocanhz

Tháng 9, 2026

Về quyển này

Phần lớn tài liệu kỹ thuật dạy bạn con đường thẳng: gõ lệnh này, được kết quả kia. Quyển này đi đường ngược lại. Nó chỉ viết về những lúc con đường thẳng ấy gãy, vì đó mới là chỗ người ta thật sự học được nghề.

Toàn bộ sự cố trong sách là có thật, lấy từ nhật ký một lần đưa dự án lên máy chủ thật: một trang xem phim gồm backend Spring Boot và frontend Next.js, đóng gói bằng Docker, phát hành qua GitHub Actions. Không có ví dụ nào được bịa ra cho đẹp bài. Thông báo lỗi trong sách là thông báo lỗi thật, đã chép nguyên văn.

Tôi viết ở ngôi thứ nhất, giọng của người chịu trách nhiệm cho kết quả cuối chứ không phải người đứng ngoài giảng giải. Vì bài học thật sự của mỗi sự cố hiếm khi nằm ở câu lệnh sửa lỗi. Nó nằm ở câu hỏi: *vì sao chúng ta không thấy sớm hơn?*

Cách đọc

Mỗi sự cố được đánh số **SC-*nn*** và trình bày theo cùng một khuôn: triệu chứng, thông báo lỗi nguyên văn, chẩn đoán, cách khắc phục, rồi nguyên tắc rút ra. Bạn có thể đọc tuần tự như một quyển sách, hoặc tra theo Phụ lục A khi đang gặp đúng lỗi đó.

- **Phần I** nêu năm nguyên tắc chi phối toàn bộ phần còn lại.
- **Phần II–III** về Git và GitHub CLI: nơi sai lầm còn rẻ.
- **Phần IV–V** về Docker và phát hành: nơi sai lầm bắt đầu đắt.
- **Phần VI** về lỗi trong chính phần mềm bạn viết — phần nhiều người bỏ qua.

Một lời thú nhận

Trong số các sự cố dưới đây, có vài cái do chính tôi gây ra bằng cách khẳng định một điều mà tôi chưa kiểm chứng. Tôi giữ nguyên chúng trong sách, kể cả phần khó coi. Một quyển sổ sự cố mà tác giả lúc nào cũng đúng thì không phải sổ sự cố — nó là quảng cáo.

Mục lục

Về quyển này	2
Cách đọc	2
Một lời thú nhận	2
1. Năm nguyên tắc	6
1.1 Lỗi im lặng đắt hơn lỗi ồn ào	6
1.2 Đo, đừng đoán	6
1.3 Thông báo lỗi là giao diện người dùng	6
1.4 Việc chưa merge là việc chưa xong	7
1.5 Không lách hàng rào an toàn	7
2. Mô hình dữ liệu, và vì sao nó cứu bạn	9
2.1 Commit là ảnh chụp, không phải bản vá	9
2.2 Nhánh chỉ là một con trỏ	9
2.3 Ba vùng cần phân biệt	9
3. Sổ sự cố Git	10
4. Xác thực và phạm vi quyền	13
5. Vòng đời một pull request	14
6. Sự cố môi trường dòng lệnh	16
7. Dockerfile: cấu trúc và thứ tự	18
7.1 Mỗi lệnh là một lớp, và lớp thì không xoá được	18
7.2 Thứ tự quyết định bộ nhớ đệm	18
7.3 Nhiều giai đoạn: dừng ở một nơi, chạy ở nơi khác	18
7.4 Tập .dockerignore: thứ hay bị quên nhất	19
8. Người dùng, quyền, và volume	20
9. Lúc dựng và lúc chạy: ranh giới hay bị xoá nhoà	21
10. Ảnh chạy trên nhiều kiến trúc	24
10.1 Ảnh đa kiến trúc thật ra là gì	24
10.2 Dựng thật hay giả lập?	26
11. Registry và quyền hiển thị	28
12. Thiết kế thông báo lỗi	30

Phụ lục A. Bảng tra cứu nhanh	32
Phụ lục B. Danh mục kiểm tra trước khi phát hành	33
Trước khi hợp nhất	33
Trước khi tin rằng ảnh dùng được	33
Trước khi nói người khác cài được	33
Trước khi nói xong	33
Phụ lục C. Lệnh dùng thường xuyên	34
Git	34
GitHub CLI	34
Docker	34
Kiểm registry mà không cần đăng nhập	34

PHẦN I

Năm nguyên tắc

Những điều tôi ước mình biết trước khi gõ lệnh đầu tiên

1. Năm nguyên tắc

Trước khi đi vào từng lệnh, tôi muốn nói về năm điều chi phối mọi thứ còn lại. Chúng không phải mẹo vặt. Chúng là cách tôi quyết định nên tin cái gì, nên kiểm cái gì, và khi nào thì được phép nói *xong rồi*.

1.1 Lỗi im lặng đắt hơn lỗi ồn ào

Trong dự án này có một đoạn mã nuốt mọi lỗi 4xx từ dịch vụ bên ngoài rồi trả về danh sách rỗng. Ý định ban đầu rất hợp lý: khi không tìm thấy bản ghi thì trả rỗng, đừng làm sập trang.

Nhưng *không tìm thấy* (404) và *khoá của bạn bị từ chối* (401) là hai chuyện khác nhau hoàn toàn. Gộp chúng lại, giao diện hiện ra dòng chữ hiền lành “**o phim**”. Người dùng nhìn thấy con số 0 và đi chỉnh bộ lọc tìm kiếm — trong khi lỗi thật nằm ở một cái khoá đặt sai chỗ, cách đó ba tầng.

NGUYÊN TẮC 1

Một hệ thống im lặng khi hỏng sẽ tiêu tốn thời gian của con người nhiều hơn bất kỳ lỗi nào chịu kêu lên. Hãy nuốt đúng một trường hợp bạn thật sự hiểu, và để mọi trường hợp còn lại kêu to.

1.2 Đo, đừng đoán

Khi thêm hỗ trợ kiến trúc ARM cho ảnh Docker, tôi viết trong mô tả thay đổi rằng *cả ba ảnh nền đều có sẵn bản arm64*. Tôi không kiểm. Tôi nhớ là như vậy, và điều đó nghe rất hợp lý.

Bản Alpine của Eclipse Temurin chỉ phát hành cho amd64. Lần dựng đầu tiên hỏng đúng ở đó. Nếu tôi bỏ ra ba mươi giây gọi API của registry, tôi đã biết trước — và đã không phải sửa sau khi mọi thứ đã lên nhánh chính.

Chuyện này lặp lại lần thứ hai trong cùng dự án, theo hướng ngược lại. Khi chọn ảnh thay thế, trực giác nói *Alpine thì nhẹ hơn*. Tôi đo thử: bản Alpine của một nhà phát hành nặng **145 MB**, trong khi bản nền Ubuntu của nhà khác chỉ **89 MB**, vì bản Alpine kia đóng gói cả bộ công cụ biên dịch. Nếu đi theo trực giác, tôi đã làm ảnh phồng lên gấp ba.

Và lần thứ ba, ngay khi dựng chính quyển sách này: tôi chọn một phông chữ rồi kiểm bằng cách đo bề rộng chuỗi. Phép đo trả về số đẹp, nên tôi tin là xong. Trang bìa in ra thì chữ có dấu biến thành ô vuông — bề rộng vẫn có số kể cả khi phông không hề chứa ký tự đó. Tôi đã đo, nhưng đo nhầm thứ.

NGUYÊN TẮC 2

Trong hạ tầng, trực giác là giả thuyết chứ không phải dữ kiện. Cái giá của một phép đo luôn rẻ hơn cái giá của một lần dựng hỏng. Nhưng hãy kiểm cả chính phép đo: một phép đo sai còn nguy hiểm hơn không đo, vì nó cho bạn sự tự tin.

1.3 Thông báo lỗi là giao diện người dùng

Trang lỗi của chúng tôi luôn khuyên: *Kiểm tra backend đã chạy tại localhost:8080 chưa*. Câu đó đúng vào ngày nó được viết, trên máy của lập trình viên. Sau khi đóng gói Docker và đưa lên máy chủ, backend chạy hoàn toàn bình thường ở một địa chỉ khác, còn thứ hỏng là dịch vụ bên ngoài.

Kết quả: mỗi lần có sự cố, lời khuyên đầu tiên người dùng đọc được lại dẫn họ đi sai hướng. Thông báo lỗi không phải là chỗ đổ ngoại lệ ra cho xong. Nó là màn hình mà người dùng gặp đúng vào lúc họ bối rối

nhất.

NGUYÊN TẮC 3

Hãy đọc thông báo lỗi của bạn bằng con mắt của người chưa từng đọc mã nguồn. Nếu nó không nói được *phải làm gì tiếp theo*, nó chưa xong.

1.4 Việc chưa merge là việc chưa xong

Tôi từng sửa xong một lỗi, mở pull request, viết mô tả cẩn thận, rồi báo lại rằng đã xử lý. Người đối diện trả lời gọn lỏn: *chưa xong lỗi kia*. Họ đúng. Nhánh chính vẫn giữ nguyên đoạn mã hỏng. Pull request đang mở là một **đề nghị**, không phải một thay đổi.

Đây là loại nhầm lẫn rất dễ mắc khi bạn đo tiến độ bằng công sức đã bỏ ra thay vì bằng trạng thái của hệ thống. Câu hỏi đúng không bao giờ là *tôi đã làm gì*, mà là *hệ thống bây giờ đang ở trạng thái nào*.

NGUYÊN TẮC 4

Chỉ có hai trạng thái: đã vào nhánh chính và đang chạy, hoặc chưa. Mọi thứ ở giữa đều là việc dở dang, dù bạn đã bỏ vào đó bao nhiêu công.

1.5 Không lách hàng rào an toàn

Vài lần trong dự án, một thao tác bị chặn: đọc tệp chứa khoá bí mật, chuyển một gói riêng tư thành công khai. Trong cả hai trường hợp, tôi dừng lại và báo cho người có quyền quyết định, thay vì tìm đường vòng.

Điều thú vị là việc chẩn đoán vẫn hoàn thành được mà không cần vượt rào. Để biết một khoá có đặt sai ô hay không, tôi không cần đọc giá trị của nó — chỉ cần xét **dạng** của nó. Khoá `v3` là 32 ký tự hệ mười sáu; token `v4` là chuỗi JWT bắt đầu bằng `eyJ`. Bấy nhiêu là đủ để kết luận, mà không có bí mật nào rời khỏi chỗ của nó.

NGUYÊN TẮC 5

Khi gặp hàng rào, hãy hỏi liệu có cách đạt mục tiêu mà không cần tháo rào hay không. Thường là có, và cách đó còn tốt hơn. Nếu không có, hãy để người chịu trách nhiệm quyết định — đừng tự quyết thay họ.

PHẦN II

Git

Nơi sai lầm còn rẻ

2. Mô hình dữ liệu, và vì sao nó cứu bạn

Hầu hết nỗi sợ khi dùng Git đến từ việc coi nó như một cỗ máy bí ẩn. Thực ra mô hình của nó nhỏ đến bất ngờ, và hiểu nó là cách duy nhất để sửa sai một cách bình tĩnh.

2.1 Commit là ảnh chụp, không phải bản vá

Nhiều người hình dung commit là *tập hợp các thay đổi*. Không phải. Mỗi commit là một **ảnh chụp toàn bộ cây thư mục** tại một thời điểm, cộng với con trỏ tới commit cha. Phần khác biệt mà bạn thấy khi chạy `git diff` là thứ Git tính ra khi so hai ảnh chụp, chứ không phải thứ nó lưu.

Hệ quả rất thực dụng: một commit đã tạo ra thì nội dung của nó không mất, kể cả khi không còn nhánh nào trỏ tới. Đó là nền tảng của mọi thao tác cứu hộ ở chương sau.

2.2 Nhánh chỉ là một con trỏ

Một nhánh trong Git là một tệp văn bản chứa mã băm của một commit. Chỉ vậy thôi. Tạo nhánh là ghi 41 byte; xoá nhánh là xoá 41 byte đó. Điều này giải thích vì sao tạo nhánh trong Git gần như miễn phí, và vì sao *xoá nhầm nhánh* hiếm khi là thảm họa — commit vẫn còn nguyên, chỉ là không còn tên gọi.

Ba lệnh cơ bản, và điều chúng thật sự làm

```
git branch tinh-nang-moi      # tạo con trỏ moi tại HEAD
git switch tinh-nang-moi     # doi HEAD sang trỏ vào nhánh mới
git switch -c tinh-nang-moi   # gộp hai bước trên làm một
```

2.3 Ba vùng cần phân biệt

Vùng	Là gì	Lệnh nhìn vào nó
Cây làm việc	Tập thật trên đĩa, thứ trình soạn thảo đang mở	<code>git status</code>
Vùng đệm (index)	Nội dung đã chọn cho commit kế tiếp	<code>git diff --staged</code>
HEAD	Ảnh chụp của commit hiện tại	<code>git show HEAD</code>

Gần như mọi lệnh Git khiến bạn bối rối đều trở nên dễ hiểu khi bạn hỏi: lệnh này đụng vào vùng nào trong ba vùng trên? `git reset --soft` chỉ dời HEAD. `--mixed` dời HEAD và xoá vùng đệm. `--hard` dời cả ba, và đó là lệnh duy nhất trong nhóm có thể làm mất công việc chưa commit.

3. Số sự cố Git

Ba sự cố dưới đây đều xảy ra trong một tuần triển khai. Không cái nào là lỗi hiếm gặp; tất cả đều thuộc loại ai làm lâu cũng gặp ít nhất một lần.

SC-01 · Commit rơi thẳng vào nhánh chính

TRIỆU CHỨNG

Sửa xong một tệp, commit, rồi mới nhận ra `git branch --show-current` trả về `main`. Thay đổi đã nằm trong lịch sử của nhánh chính, chưa qua rà soát nào.

Đây là sự cố lành tính **nếu bạn phát hiện trước khi đẩy lên máy chủ**. Vì nhánh chỉ là con trỏ, việc cần làm là tạo một con trỏ mới tại chỗ đang đứng, rồi kéo con trỏ `main` lùi lại.

Cách gỡ, theo đúng thứ tự

```
git branch tinh-nang-moi      # dat ten cho cong viec vua commit
git reset --hard <ma-commit-truoc-do>
git switch tinh-nang-moi
```

```
# Xac nhan main da khop lai voi may chu:
git rev-parse main origin/main | uniq -c    # ra mot dong = khop
```

Thứ tự quan trọng. Phải tạo nhánh **trước** khi reset; làm ngược lại thì commit mất tên và bạn phải đi tìm nó trong `git reflog`. Và chỉ dùng cách này khi chưa đẩy lên: một khi người khác đã kéo về, việc viết lại lịch sử chuyển gánh nặng sang họ.

PHÒNG NGỪA

Bật bảo vệ nhánh chính trên máy chủ, để chính máy chủ từ chối. Một quy ước chỉ tồn tại trong đầu người ta là quy ước sẽ bị quên vào lúc bận nhất.

SC-02 · Tệp đã xoá vẫn sống trên máy chủ

TRIỆU CHỨNG

Đổi tên một đường dẫn từ `/quan-tri` sang `/cms`, triển khai, nhưng đường dẫn cũ vẫn trả về mã 200. Trên máy phát triển thì không còn dấu vết nào.

Kịch bản triển khai đóng gói mã nguồn bằng `tar` rồi giải nén đè lên thư mục trên máy chủ. Giải nén đè chỉ **ghi thêm và ghi đè** — nó không bao giờ xoá. Mọi tệp bạn đã xoá ở máy mình vẫn nằm nguyên ở đích, và với một khung như Next.js vốn sinh đường dẫn từ cấu trúc thư mục, tệp cũ còn nghĩa là đường dẫn cũ còn.

Vì sao bản triển khai cũ vẫn còn sống

```
# Sai: chỉ phu len, khong bao gio don
tar -xzf ban-moi.tar.gz -C /opt/ung-dung

# Dung: don sach thu muc ma nguon truoc khi giai nen
rm -rf /opt/ung-dung/backend /opt/ung-dung/frontend
tar -xzf ban-moi.tar.gz -C /opt/ung-dung
```

BÀI HỌC

Bất kỳ cơ chế triển khai nào hoạt động theo kiểu *phủ lên* đều mang sẵn khuyết tật này. Hãy hỏi: nếu tôi xoá một tệp, cơ chế này có xoá nó ở đích không? Nếu câu trả lời là không, bạn đang tích lũy một tầng rác âm thầm.

SC-03 · Ba công cụ sửa sai, và khi nào dùng cái nào

Người mới thường học `reset --hard` rồi dùng nó cho mọi tình huống. Đó là con dao sắc nhất trong hộp, và cũng là con dễ đứt tay nhất.

Công cụ	Làm gì	Dùng khi
<code>git reset</code>	Dời con trỏ nhánh, có thể bỏ công việc chưa commit	Chưa đẩy lên, muốn lịch sử sạch
<code>git revert</code>	Tạo commit mới đảo ngược một commit cũ	Đã đẩy lên, người khác đã có bản đó
<code>git reflog</code>	Nhật ký mọi vị trí HEAD từng đứng	Đã lỡ tay và cần tìm lại commit

`git reflog` là lưới an toàn ít người biết đến. Nó ghi lại mọi lần HEAD dịch chuyển trong khoảng ba mươi ngày, kể cả những commit không còn nhánh nào trỏ tới. Gần như mọi tình huống *tôi làm mất commit* rồi đều giải được bằng nó.

Cứu một commit tưởng đã mất

```
git reflog # tìm dòng có ma commit cần cứu
git switch -c cuu-ho <ma-commit> # dat lai cho no mot cai ten
```

PHẦN III

GitHub CLI

Khi dòng lệnh gặp dịch vụ đám mây

4. Xác thực và phạm vi quyền

gh là GitHub gói trong một lệnh. Nó rút ngắn rất nhiều thao tác, nhưng mang theo một khái niệm mà giao diện web che đi mất: **phạm vi quyền** của thẻ xác thực.

Thẻ xác thực của bạn không phải chia khoá vạn năng. Nó mang một danh sách phạm vi cụ thể, và một thao tác nằm ngoài danh sách đó sẽ bị từ chối — kể cả khi tài khoản của bạn thừa quyền làm việc ấy trên web.

SC-04 · Thiếu phạm vi quyền

TRIỆU CHỨNG

Gọi API qua gh và nhận về lỗi 403 dù bạn là chủ sở hữu:

```
You need at least read:packages scope to get a package. (HTTP 403)
```

Thông báo này thuộc loại tử tế hiếm gặp: nó nói thẳng thiếu phạm vi nào. Cách sửa là xin bổ sung, và gh sẽ mở trình duyệt cho bạn duyệt lại.

Bổ sung phạm vi cho thẻ hiện tại

```
gh auth refresh --scopes read:packages,write:packages
gh auth status # kiểm lại danh sách phạm vi hiện có
```

BÀI HỌC

Khi một thao tác bị từ chối, hãy phân biệt ba nguyên nhân trước khi sửa: tài khoản không đủ quyền, thẻ không đủ phạm vi, hay đối tượng không tồn tại. Ba nguyên nhân này có cách xử lý khác hẳn nhau, mà đều hiện ra dưới dạng một con số 403 hoặc 404 giống nhau.

5. Vòng đời một pull request

Từ dòng lệnh, cả vòng đời của một thay đổi gói gọn trong bốn lệnh. Cái bẫy không nằm ở lệnh nào khó, mà ở chỗ dễ tưởng mình đã đi hết vòng.

Toàn bộ vòng đời

```
git switch -c sua-loi-abc
git commit -am "Mô tả ngắn gọn việc đã làm"
git push -u origin sua-loi-abc
gh pr create --base main --title "... " --body-file -
gh pr merge <so> --squash --delete-branch
```

SC-05 · Pull request mở, nhưng lỗi vẫn còn

TRIỆU CHỨNG

Bạn báo cáo đã sửa xong. Người khác kiểm tra và thấy lỗi vẫn nguyên vẹn trên nhánh chính.

Tôi đã mắc đúng lỗi này. Pull request được mở, mô tả viết kỹ, kiểm thử chạy sạch — nhưng nhánh chính chưa hề thay đổi. Một pull request đang mở là lời đề nghị chờ người khác đồng ý, không phải một sự thật đã xảy ra.

Kiểm tra trạng thái thật thay vì trí nhớ

```
gh pr view <so> --json state,mergedAt
gh run list --limit 3          # nhánh chính đã chạy lại chưa?
```

NGUYÊN TẮC

Trước khi nói xong rồi, hãy chạy một lệnh đọc trạng thái thật của hệ thống. Trí nhớ về việc mình vừa làm không phải bằng chứng.

SC-06 · Không có kiểm thử tự động nào chạy trên pull request

TRIỆU CHỨNG

gh pr checks trả về:

```
no checks reported on the 'ten-nhanh' branch
```

Nguyên nhân gần như luôn nằm ở phần khai báo sự kiện kích hoạt của quy trình. Một quy trình chỉ khai báo on: push: branches: [main] sẽ **không bao giờ** chạy cho pull request. Nghĩa là mọi thay đổi vào quy trình đó chỉ được kiểm chứng sau khi đã hợp nhất — đúng lúc sửa sai đắt nhất.

Cho quy trình chạy cả trên pull request

```
on:
  push:
    branches: [main]
  pull_request:      # thêm dòng này để PR cũng được kiểm
    branches: [main]
```

BÀI HỌC

Một pull request màu xanh không có nghĩa là đã kiểm thử. Nó có thể chỉ có nghĩa là chưa ai kiểm gì cả. Hãy phân biệt *không có lỗi* với *không có phép thử* nào.

6. Sự cố môi trường dòng lệnh

Hai sự cố dưới đây không thuộc về Git hay Docker. Chúng thuộc về cái vỏ bọc quanh mọi thứ: trình thông dịch dòng lệnh. Chúng chiếm một lượng thời gian hoang phí lớn hơn nhiều so với mức người ta thừa nhận.

SC-07 · `curl` trong PowerShell không phải `curl`

TRIỆU CHỨNG

Sao chép một lệnh `curl` từ tài liệu, dán vào PowerShell, và nó hỏi ngược lại bạn một tham số:

```
cmdlet Invoke-WebRequest at command pipeline position 1
Supply values for the following parameters:
Uri:
```

Trong Windows PowerShell, `curl` là bí danh của `Invoke-WebRequest` — một lệnh hoàn toàn khác, với bộ tham số hoàn toàn khác. Cờ `-s` của `curl` thật không tồn tại ở đây, nên chuỗi địa chỉ không gắn được vào tham số nào và lệnh quay ra hỏi bạn.

Ký tự `&` trong địa chỉ còn gây rắc rối thứ hai: PowerShell coi nó là ký tự dành riêng. Cách chữa dứt điểm là gọi thẳng chương trình thật bằng đuôi `.exe`, và luôn bọc địa chỉ trong dấu nháy.

Gọi đúng chương trình

```
# Sai trong PowerShell:
curl -s "https://vi.du/api?a=1&b=2"

# Đúng:
curl.exe -s "https://vi.du/api?a=1&b=2"
```

SC-08 · Dấu nhắc đang chờ nuốt mất lệnh kế tiếp

TRIỆU CHỨNG

Sau sự cố trên, bạn gõ lệnh tiếp theo và nó không chạy — thay vào đó bạn nhận một lỗi hoàn toàn không liên quan:

```
Uri: gh auth refresh --scopes read:packages
curl : Cannot find drive. A drive with the name 'https' does not exist.
```

Cửa sổ dòng lệnh vẫn đang treo ở dấu nhắc `Uri:` từ lệnh hỏng trước đó. Lệnh `gh` bạn vừa gõ không được thực thi; nó bị nhận làm **giá trị** cho tham số `Uri`. Trong ba mươi giây, bạn tin rằng mình đã chạy `gh auth refresh` trong khi thực tế nó chưa từng chạy.

BÀI HỌC

Khi kết quả không khớp với việc bạn nghĩ mình vừa làm, hãy nghi ngờ **trạng thái của cửa sổ dòng lệnh** trước khi nghi ngờ hệ thống. Bấm `Ctrl+C`, xác nhận đã về dấu nhắc bình thường, rồi mới làm tiếp.

PHẦN IV

Dockerfile và ảnh

Nơi sai lầm bắt đầu đặt

7. Dockerfile: cấu trúc và thứ tự

Một Dockerfile trông như một kịch bản shell, và đó chính là lý do người ta viết nó sai. Nó không phải kịch bản. Nó là **bản mô tả một chuỗi lớp**, và thứ tự các dòng quyết định cả tốc độ dựng lẫn dung lượng cuối cùng.

7.1 Mỗi lệnh là một lớp, và lớp thì không xóa được

Mỗi RUN, COPY, ADD sinh ra một lớp mới xếp lên lớp trước. Điểm cốt tử: lớp sau **không thể xóa dữ liệu** của lớp trước, chỉ có thể che nó đi. Tải về một tệp 200 MB ở một dòng rồi xóa nó ở dòng sau, ảnh của bạn vẫn nặng thêm 200 MB — tệp ấy vẫn nằm trong lớp cũ, chỉ không còn thấy được.

Cùng một việc, hai dung lượng khác nhau

```
# Sai: tệp tải về nằm mãi trong lớp thu nhất
RUN wget https://vi.du/bo-cai.tar.gz
RUN tar -xzf bo-cai.tar.gz && rm bo-cai.tar.gz

# Dung: tải, dung, và dọn trong cùng một lớp
RUN wget https://vi.du/bo-cai.tar.gz \
    && tar -xzf bo-cai.tar.gz \
    && rm bo-cai.tar.gz
```

7.2 Thứ tự quyết định bộ nhớ đệm

Docker dùng lại một lớp khi lệnh sinh ra nó và mọi thứ nó phụ thuộc đều không đổi. Một lớp bị coi là **bản** thì mọi lớp sau nó cũng bản theo. Từ đó ra một quy tắc đơn giản: **đặt thứ ít thay đổi lên trước**.

Trong một dự án Java hay Node, phần khai báo phụ thuộc thay đổi vài tuần một lần, còn mã nguồn thay đổi vài phút một lần. Nếu bạn chép cả thư mục vào rồi mới tải phụ thuộc, mỗi lần sửa một dòng mã là tải lại toàn bộ thư viện.

Sắp xếp để bộ nhớ đệm còn tác dụng

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build

# Chỉ khai báo phụ thuộc trước: đôi rat ít, nên lớp này dựng lại được lâu
COPY pom.xml .
RUN mvn -B dependency:go-offline

# Mã nguồn vào sau: sửa mà không làm mất lớp tải thư viện ở trên
COPY src ./src
RUN mvn -B clean package -DskipTests
```

7.3 Nhiều giai đoạn: dựng ở một nơi, chạy ở nơi khác

Bộ công cụ biên dịch nặng hơn thứ chạy thật rất nhiều. Ảnh Maven kèm JDK khoảng 600 MB; một JRE đủ để chạy chỉ khoảng 50 MB. Không có lý gì để mang bộ biên dịch lên máy chủ.

Nhiều giai đoạn giải quyết chuyện này: giai đoạn đầu dựng, giai đoạn cuối chỉ **chép kết quả** sang một ảnh nền gọn. Mọi thứ ở giai đoạn đầu bị bỏ lại.

Giai đoạn chạy chỉ nhận đúng thứ cần

```
FROM bellsoft/liberica-openjre-alpine:17
WORKDIR /app
COPY --from=build /build/target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

7.4 Tập `.dockerignore`: thứ hay bị quên nhất

Không có tập này, `COPY . .` sẽ nhồi cả `node_modules`, `target`, `.git` và mọi tệp cấu hình cục bộ vào ngữ cảnh dựng. Hậu quả không chỉ là chậm: một tệp `.env` lọt vào ảnh là bí mật đã rời khỏi máy bạn.

Một `.dockerignore` tối thiểu

```
.git
node_modules
target
.next
*.log
.env
.env.*
```

8. Người dùng, quyền, và volume

Container chạy bằng `root` là mặc định, và là mặc định nên đối. Nhưng đối không đúng chỗ thì sinh ra một lỗi chỉ lộ ra lúc chạy thật — loại lỗi tệ nhất.

SC-09 · Container không ghi được vào volume của chính nó

TRIỆU CHỨNG

Ảnh dựng xong, container khởi động, rồi chết ngay khi ghi tệp đầu tiên:

```
java.nio.file.AccessDeniedException: /app/data/settings.json
```

Chuỗi nhân quả ở đây có ba mắt, và phải hiểu cả ba mới sửa đúng. Khi Docker tạo một volume mới, nó **kế thừa quyền sở hữu từ thư mục tương ứng trong ảnh**. Nếu thư mục đó thuộc `root` mà tiến trình lại chạy bằng người dùng thường, tiến trình không ghi được. Và vì volume chỉ lấy quyền *một lần lúc tạo*, sửa Dockerfile rồi dựng lại vẫn không đủ — volume cũ giữ nguyên quyền sai.

Thứ tự đúng trong Dockerfile

```
RUN addgroup -S raphim && adduser -S raphim -G raphim

# Phai tao thu muc VA giao quyen TRUOC khi doi user.
# Lam sau dong USER thi khong con quyen ma chown nua.
RUN mkdir -p /app/data && chown -R raphim:raphim /app/data

USER raphim
COPY --from=build --chown=raphim:raphim /build/target/*.jar app.jar
```

Và phải bỏ volume cũ đi

```
docker compose down
docker volume rm ten-du-an-du-lieu    # volume cu giu quyen sai
docker compose up -d                  # volume moi ke thua quyen dung
```

BÀI HỌC

Volume là trạng thái tồn tại lâu hơn ảnh. Khi sửa một lỗi liên quan tới quyền hay dữ liệu ban đầu, hãy hỏi thêm: *có trạng thái cũ nào còn sót lại làm bản sửa của tôi vô hiệu không?*

9. Lúc dựng và lúc chạy: ranh giới hay bị xoá nhoà

Đây là nhóm lỗi tôi thấy gây thiệt hại nhiều nhất, vì nó không làm gì sập. Ảnh dựng thành công, container khởi động khoẻ mạnh, và sản phẩm thì sai.

SC-10 · Địa chỉ mạng nội bộ bị nướng vào ảnh

TRIỆU CHỨNG

Triển khai xong, trang tải được nhưng trắng trơn. Không lỗi nào trong log. Kích thước trang trả về 39 KB thay vì 300 KB — bộ khung có, dữ liệu không.

Các khung giao diện hiện đại **thay thế biến môi trường ngay lúc dựng** cho những biến dành cho trình duyệt. Với Next.js đó là tiền tố `NEXT_PUBLIC_`. Giá trị bạn đưa vào lúc `docker build` không còn là biến nữa; nó trở thành chuỗi ký tự nằm cứng trong tệp JavaScript.

Chúng tôi đưa địa chỉ mạng nội bộ `192.168.100.169:8081` vào lúc dựng. Ảnh chạy được trên đúng mạng đó, và vô dụng ở mọi nơi khác. Tệ hơn: chính container lại không gọi được địa chỉ LAN ấy, nên phần kết xuất phía máy chủ treo và trả ra trang rỗng.

Lời giải là tách hai đường gọi, vì chúng thật sự là hai đường khác nhau:

Biến	Ai dùng	Giá trị đúng	Nằm trong ảnh?
<code>NEXT_PUBLIC_API_BASE_URL</code>	Trình duyệt người xem	Đường tương đối, để proxy lo	Có, không tránh được
<code>API_INTERNAL_URL</code>	Tiến trình trong container	<code>http://backend:8080</code>	Không, đọc lúc chạy

Cách chắc chắn nhất là **đừng để trình duyệt biết địa chỉ backend**. Cho mọi yêu cầu đi qua một đường dẫn tương đối trên chính máy chủ web, rồi để máy chủ đó chuyển tiếp. Lúc ấy ảnh không chứa địa chỉ nào, và dùng được ở mọi nơi.

SC-11 · Ảnh nền thiếu công cụ mà *healthcheck* cần

TRIỆU CHỨNG

Ảnh dựng xanh, container chạy, nhưng mãi bị đánh dấu `unhealthy`. Dịch vụ phụ thuộc nó thì không bao giờ khởi động.

`HEALTHCHECK` chạy lệnh **bên trong** container. Nếu bạn viết `wget` hay `curl` mà ảnh nền không có, phép kiểm luôn thất bại — dù ứng dụng hoàn toàn bình thường. Ảnh nền Alpine có sẵn `wget` nhờ `busybox`; ảnh nền Debian hay Ubuntu tối giản thì **không có cả hai**.

Đây là lý do rất thực tế để cân nhắc kỹ khi đổi ảnh nền. Trong dự án này, chuyện đó quyết định lựa chọn ở chương sau: giữ nền Alpine để không phải viết lại `HEALTHCHECK` và lệnh tạo người dùng.

Cùng một ý định, hai nền khác nhau

```
# Trên nền Alpine: dùng ngay
HEALTHCHECK CMD wget -qO- http://127.0.0.1:8080/actuator/health || exit 1

# Trên nền Ubuntu: phải cài thêm, hoặc doi phép kiểm
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

BÀI HỌC

Đổi ảnh nền không chỉ đổi dung lượng. Nó đổi cả tập lệnh có sẵn, cú pháp tạo người dùng, và thư viện hệ thống. Hãy đọc lại toàn bộ Dockerfile mỗi lần đổi dòng **FROM**.

PHẦN V

Đa kiến trúc và phát hành

Khi ảnh của bạn rời khỏi máy bạn

10. Ảnh chạy trên nhiều kiến trúc

Chừng nào ảnh chỉ chạy trên máy bạn, kiến trúc không phải vấn đề. Khoảnh khắc bạn nói *ai cũng cài được*, nó trở thành vấn đề trung tâm — vì máy Mac dùng chip Apple, máy chủ giá rẻ dùng ARM, và cả hai đều không chạy được ảnh amd64 một cách tử tế.

10.1 Ảnh đa kiến trúc thật ra là gì

Một nhãn như `latest` không trở tới một ảnh. Nó trở tới một **bảng chỉ mục** liệt kê nhiều ảnh, mỗi ảnh kèm nền tảng của nó. Khi bạn kéo về, Docker đọc bảng đó và chọn dòng khớp với máy bạn. Hiểu điều này là hiểu tất cả các lỗi còn lại trong chương.

Xem một nhãn thật sự chứa những nền tảng nào

```
docker buildx imagetools inspect ghcr.io/chu-so-huu/ten-anh:latest
```

SC-12 · Ảnh nền không có kiến trúc bạn cần

TRIỆU CHỨNG

Việc dựng cho ARM hỏng ngay ở bước kéo ảnh nền:

```
ERROR: failed to solve: eclipse-temurin:17-jre-alpine: failed to resolve
source metadata for docker.io/library/eclipse-temurin:17-jre-alpine:
no match for platform in manifest: not found
```

Câu `no match for platform in manifest` nói rất chính xác: bảng chỉ mục của ảnh nền không có dòng nào cho nền tảng bạn yêu cầu. Không phải mạng lỗi, không phải gõ sai tên. Ảnh đó **chưa từng được phát hành** cho kiến trúc ấy.

Đây là sự cố do tôi gây ra, theo cách đáng nói: tôi đã viết trong mô tả thay đổi rằng cả ba ảnh nền đều có bản arm64, dựa vào trí nhớ. Bản Alpine của Eclipse Temurin chỉ có amd64. Ba mươi giây tra cứu đã tránh được cả lần dựng hỏng và một khẳng định sai trong tài liệu.

Kiểm nền tảng trước khi tin

```
docker buildx imagetools inspect eclipse-temurin:17-jre-alpine
```

```
# Hoac khong can Docker, goi thang registry:
curl.exe -s "https://auth.docker.io/token?service=registry.docker.io
&scope=repository:library/eclipse-temurin:pull"
```

Kết quả đo được khi chọn ảnh thay thế, xếp theo dung lượng lớp nền đã nén cho arm64:

Ảnh nền	Kiến trúc	arm64
eclipse-temurin:17-jre-alpine	chỉ amd64	—
bellsoft/liberica-openjre-alpine:17	amd64, arm64	44 MB
azul/zulu-openjdk-alpine:17-jre	amd64, arm64	72 MB
eclipse-temurin:17-jre-noble	sáu kiến trúc	89 MB
amazoncorretto:17-alpine	amd64, arm64	145 MB

Bảng này lật một giả định phổ biến: *Alpine thì nhẹ hơn*. Bản Alpine nặng nhất bảng, vì nó đóng gói cả JDK thay vì chỉ JRE. Nếu tôi chọn theo trực giác thay vì theo số đo, ảnh đã phồng lên gấp ba.

SC-13 · Hai máy dựng ghi đè nhãn của nhau

TRIỆU CHỨNG

Quy trình báo thành công, nhưng nhãn `latest` chỉ chứa một kiến trúc. Kiến trúc kia biến mất không dấu vết.

Đây là cái bẫy tinh vi nhất của việc dựng đa kiến trúc trên nhiều máy song song. Nếu mỗi máy tự gắn nhãn `latest` cho bản của mình, máy nào xong sau sẽ **ghi đè** lên nhãn của máy xong trước. Nhãn là một con trỏ, và con trỏ chỉ trỏ được vào một chỗ.

Đáng lo là mọi thứ vẫn xanh. Không có lỗi nào để đọc. Bạn chỉ phát hiện khi có người trên kiến trúc kia báo không chạy được.

Cách làm đúng gồm hai bước tách biệt. Bước dựng đẩy từng bản lên registry **không kèm nhãn**, chỉ định danh bằng digest. Bước gộp nối các digest ấy thành một bảng chỉ mục rồi mới gắn nhãn:

Đẩy theo digest, rồi mới gộp và gắn nhãn

```
# Bước 1 - trên từng máy, đẩy lên mà không đặt nhãn
- uses: docker/build-push-action@v6
  with:
    platforms: linux/${{ matrix.platform.name }}
    outputs: type=image,name=${{ env.ANH }},push-by-digest=true,
             name-canonical=true,push=true

# Bước 2 - một việc riêng: gộp các digest lại rồi mới gắn nhãn
- run: |
    docker buildx imagetools create
      $(jq -cr '.tags | map("-t " + .) | join(" ")'
        <<< "$DOCKER_METADATA_OUTPUT_JSON")
      $(printf '${{ env.ANH }}@sha256:%s ' *)
```

BÀI HỌC

Khi nhiều việc song song cùng ghi vào một tên chung, hãy giả định chúng sẽ ghi đè lên nhau. Cho mỗi việc ghi vào một địa chỉ riêng, rồi hợp nhất ở một bước có thứ tự rõ ràng.

SC-14 · Bộ nhớ đệm dùng chung giữa hai kiến trúc

TRIỆU CHỨNG

Thời gian dựng thất thường, có lần lâu bất thường, có lần ra ảnh sai kiến trúc một cách khó hiểu.

Lớp đã dựng cho amd64 không dùng lại được cho arm64 — chúng chứa mã máy khác nhau. Nếu hai việc dùng chung một phạm vi bộ nhớ đệm, chúng sẽ liên tục ghi đè kết quả của nhau. Cách chữa là tách phạm vi theo cả ảnh và kiến trúc:

Mỗi ảnh, mỗi kiến trúc một kho đệm riêng

```
cache-from: type=gha,scope=${{ matrix.image.name }}-${{ matrix.platform.name }}
cache-to:   type=gha,mode=max,scope=${{ matrix.image.name }}-${{ matrix.platform.name }}
```

10.2 Dựng thật hay giả lập?

Có hai đường để ra ảnh ARM. Đường thứ nhất: giả lập bằng QEMU trên máy amd64 — cấu hình một dòng, nhưng biên dịch Java hay Next.js dưới giả lập chậm tới mức dễ vượt hạn mức thời gian. Đường thứ hai: dựng trên máy ARM thật.

Với kho mã nguồn công khai, GitHub cấp runner ARM miễn phí. Chúng tôi chọn đường thứ hai và không tốn thêm gì. Đổi lại là cấu trúc quy trình phức tạp hơn — chính là hai bước ở SC-13.

CÂN NHẮC

Giả lập rẻ về cấu hình, đắt về thời gian mỗi lần chạy. Máy thật ngược lại. Với một việc chạy hàng chục lần mỗi tuần, cái đắt lặp lại luôn thắng cái đắt một lần.

11. Registry và quyền hiển thị

Dựng được ảnh chưa phải là phát hành được ảnh. Giữa hai việc đó có một bước mà quy trình xanh không nói cho bạn biết.

SC-15 · Ảnh riêng tư, nhưng không ai báo

TRIỆU CHỨNG

Quy trình thành công, ảnh có trên registry, hướng dẫn cài đặt trở đúng địa chỉ. Người khác cài thì nhận *denied*.

Ảnh do quy trình tự động đẩy lên GitHub Container Registry mặc định là **riêng tư**, bất kể kho mã nguồn công khai. Không có cảnh báo nào, vì với chính bạn mọi thứ hoạt động — thẻ xác thực của bạn có quyền.

Còn một cái bẫy nữa: quyền hiển thị của *kho mã nguồn* và của *gói* là hai thứ tách rời. Đổi kho sang công khai không đổi gói. Trong dự án này, phải ba lần thao tác mới thật sự xong, vì hộp thoại xác nhận bắt gõ lại đúng tên gói và việc bỏ ngang không để lại thông báo nào.

SC-16 · Cách kiểm mà không cần đăng nhập

Đây là kỹ thuật tôi thấy đáng giá nhất trong cả chương, vì nó trả lời đúng câu hỏi bạn cần: *người ngoài có kéo được không?* Đăng nhập rồi kéo thử không trả lời được câu đó — bạn luôn có quyền với ảnh của mình.

Registry theo chuẩn OCI cấp một thẻ đọc ẩn danh cho ảnh công khai, và từ chối với ảnh riêng tư. Chỉ cần gọi đúng một địa chỉ:

Phép thử dứt điểm, không cần đăng nhập

```
curl.exe -s "https://ghcr.io/token?service=ghcr.io
&scope=repository:chu-so-huu/ten-anh:pull"
```

```
# Co "token"          -> công khai, người ngoài kéo được
# Ra UNAUTHORIZED    -> con riêng tư
```

QUAN TRỌNG

Hãy luôn kiểm cùng một phép thử trên một ảnh **đã biết chắc là công khai** để đối chứng. Có lần tôi kết luận sai vì lệnh bóc chuỗi của tôi âm thầm trả về nguyên văn JSON lỗi, mà chuỗi đó lại dài hơn ngưỡng tôi kiểm — phép thử hỏng, chứ không phải kết quả xấu.

BÀI HỌC

Một phép kiểm chỉ đáng tin khi bạn đã thấy nó **thất bại đúng lúc cần thất bại**. Phép kiểm luôn cho kết quả tốt và phép kiểm bị hỏng nhìn giống nhau y hệt.

PHẦN VI

Lỗi trong phần mềm của bạn

Phần mà hầu hết tài liệu DevOps bỏ qua

12. Thiết kế thông báo lỗi

Ba sự cố cuối không thuộc về công cụ. Chúng thuộc về mã nguồn chúng tôi tự viết, và tôi để chúng ở cuối vì chúng tốn nhiều thời gian nhất — không phải để sửa, mà để **tìm ra**.

Bối cảnh: trang khám phá phim hiện ra o *phim*. Không lỗi, không cảnh báo. Suốt một thời gian dài không ai nghĩ đó là lỗi.

SC-17 · Nuốt cả nhóm lỗi thành kết quả rỗng

Một dòng, và ba tuần chẩn đoán sai hướng

```
// Sai: moi loi 4xx thanh danh sach rong
.onStatus(status -> status.is4xxClientError(), (req, res) -> {})

// Dung: chi 404 la "khong co ban ghi"
.onStatus(status -> status.value() == 404, (req, res) -> {})
```

Ý định ban đầu hợp lý: không tìm thấy thì trả rỗng. Nhưng `is4xxClientError()` gộp luôn 401 (khóa bị từ chối) và 429 (gọi quá nhiều) vào cùng một rổ với 404. Cả ba hiện ra giống hệt nhau trên giao diện.

Hậu quả không phải một lỗi hiển thị. Nó là **ba tuần đi sai hướng**: người dùng chỉnh bộ lọc, đổi từ khoá, nghỉ ngờ nguồn dữ liệu — trong khi nguyên nhân thật là một cái khoá đặt sai ô, và hệ thống biết điều đó ngay từ lần gọi đầu tiên nhưng đã chọn không nói.

SC-18 · Buộc hai tầng vào nhau bằng nguyên văn thông báo

TRIỆU CHỨNG

Giao diện dò chuỗi tiếng Anh trong ngoại lệ của tầng dưới để quyết định hiển thị gì:

```
// Rat de vo: sua loi nhan la am tham lam hong cho nay
const biChan = thongBao.includes("terminated the handshake")
|| thongBao.includes("I/O error");
```

Đoạn này chạy đúng — cho tới lần đầu ai đó viết lại thông báo lỗi cho dễ hiểu hơn. Lúc ấy phần gọi ý biến mất, không có lỗi biên dịch, không có kiểm thử nào đỏ. Chúng tôi đã tự tạo ra một mối phụ thuộc mà chẳng công cụ nào nhìn thấy.

Cách chữa là cho tầng dưới phát ra một **mã lỗi ổn định**, và tầng trên đọc mã đó thay vì đọc câu chữ:

Hợp đồng giữa hai tầng nên là mã, không phải văn bản

```
// Tang duoi: phan loai roi gan ma riêng
public static final String BLOCKED = "UPSTREAM_BLOCKED";

// Tang tren: doc ma, khong doc cau chu
if (loi.code !== "UPSTREAM_BLOCKED") return undefined;
```

NGUYÊN TẮC

Thông báo lỗi dành cho con người và mã lỗi dành cho máy là hai thứ khác nhau. Trộn chúng lại thì bạn không sửa được câu chữ nữa mà không sợ vỡ, và cuối cùng câu chữ sẽ cứ dở mãi.

SC-19 · Lờì khuyên đúng một lần, rồi sai mãi

TRIỆU CHỨNG

Mọi trang lỗi đều khuyên *kiểm tra backend đã chạy tại localhost:8080 chưa*, kể cả khi backend đang chạy hoàn hảo và thứ hỏng là dịch vụ bên ngoài.

Câu đó đúng vào ngày nó được viết, trên máy của lập trình viên. Sau khi đóng gói và triển khai, nó thành lời khuyên sai — dẫn người dùng đi kiểm tra đúng thứ đang hoạt động bình thường.

Cùng loại với nó là một liên kết `http://localhost:8080/swagger-ui.html` nằm trong thanh bên. Chúng tôi bỏ hẳn liên kết đó thay vì sửa: muốn nó đúng thì phải nhồi địa chỉ backend vào lúc dựng, tức là quay lại đúng SC-10.

Lời khuyên phải phụ thuộc vào nguyên nhân

```
// Chỉ khuyên kiểm backend khi thật sự không gọi được backend
<ErrorState
  message={loiNhan}
  unreachable={laKhongGoiDuoc(loi)}
  hint={goiYBiChan(loi)}
/>
```

BÀI HỌC

Mỗi câu hướng dẫn cố định trong thông báo lỗi là một giả định về môi trường. Hãy rà lại chúng mỗi lần môi trường đổi — và nếu một câu không thể luôn đúng, hãy bỏ nó đi thay vì để nó gây nhầm.

Phụ lục A. Bảng tra cứu nhanh

Tra theo triệu chứng. Số hiệu dẫn tới phần trình bày đầy đủ trong sách.

Bạn thấy gì	Nguyên nhân thật	Xem
Commit nằm sai nhánh	Quên tạo nhánh trước khi làm	SC-01
Đường dẫn cũ vẫn trả về 200 sau khi xoá	Giải nén đề không xoá tệp đã bỏ	SC-02
403 dù bạn là chủ sở hữu	Thẻ xác thực thiếu phạm vi quyền	SC-04
Đã sửa mà lỗi vẫn còn	Pull request mở nhưng chưa hợp nhất	SC-05
no checks reported	Quy trình không kích hoạt trên pull request	SC-06
Dòng lệnh hỏi Uri:	curl là bí danh của Invoke-WebRequest	SC-07
Lệnh vừa gõ không chạy	Dấu nhắc cũ nuốt mất nó	SC-08
AccessDeniedException trên volume	Volume kế thừa quyền sai, cần bỏ volume cũ	SC-09
Trang trắng, dữ liệu rỗng, log sạch	Địa chỉ nướng vào ảnh lúc dựng	SC-10
Container mãi unhealthy	Ảnh nền thiếu wget hoặc curl	SC-11
no match for platform in manifest	Ảnh nền không có kiến trúc đó	SC-12
Nhãn chỉ còn một kiến trúc	Các máy dựng ghi đè nhãn của nhau	SC-13
Thời gian dựng thất thường	Bộ nhớ đệm dùng chung giữa hai kiến trúc	SC-14
Người khác nhận denied khi kéo ảnh	Gói mặc định riêng tư, khác quyền của kho mã	SC-15
Kết quả o bản ghi nhưng nguồn vẫn tốt	Mã nuốt cả nhóm lỗi thành rỗng	SC-17
Gợi ý trên giao diện tự biến mất	Giao diện dò theo nguyên văn thông báo	SC-18
Thông báo lỗi khuyên sai hướng	Lời khuyên cố định không theo nguyên nhân	SC-19

Phụ lục B. Danh mục kiểm trước khi phát hành

Danh mục này rút ra từ chính những lần đã sai. Mỗi dòng tương ứng với một sự cố có thật ở trên.

Trước khi hợp nhất

- Đang ở nhánh phụ, không phải nhánh chính. — SC-01
- Đã chạy kiểm thử và lint tại máy, vì pull request có thể không có phép kiểm nào. — SC-06
- Mọi khẳng định trong mô tả thay đổi đều đã được kiểm, không dựa vào trí nhớ. — SC-12

Trước khi tin rằng ảnh chụp được

- `imagetools inspect` cho thấy đủ các kiến trúc dự định. — SC-13
- Không có địa chỉ mạng nội bộ nào nằm trong ảnh. — SC-10
- Thư mục dữ liệu đã `chown` trước dòng `USER`. — SC-09
- Lệnh trong `HEALTHCHECK` thật sự tồn tại trong ảnh nền. — SC-11
- `.dockerignore` có `.git`, `node_modules`, `.env`. — mục 7.4

Trước khi nói người khác cài được

- Phép thử token ẩn danh cho thấy gói công khai. — SC-15
- Đã đối chứng phép thử đó trên một ảnh biết chắc là công khai. — SC-16
- Địa chỉ ảnh trong hướng dẫn khớp với địa chỉ quy trình thật sự đẩy lên.

Trước khi nói xong

- Đã đọc trạng thái thật của hệ thống bằng một lệnh, không dựa vào trí nhớ. — SC-05
- Đã nói rõ phần nào *chưa* kiểm được, và vì sao.

Phụ lục C. Lệnh dùng thường xuyên

Git

```
git switch -c ten-nhanh      # tao va chuyen sang nhanh moi
git reflog                  # tim lai commit tuong da mat
git rev-parse main origin/main # main da khop may chu chua
git log --oneline -5        # nam commit gan nhat
```

GitHub CLI

```
gh auth status              # xem pham vi quyen hien co
gh pr create --base main --body-file -
gh pr view <so> --json state,mergedAt
gh pr merge <so> --squash --delete-branch
gh run list --limit 5
gh run view <id> --json conclusion,jobs
gh run watch <id> --exit-status
```

Docker

```
docker buildx imagetools inspect <anh>:latest
docker volume ls
docker volume rm <ten-volume>      # bo volume giu quyen sai
docker logs <container> --tail 50
docker compose ps                  # xem trang thai healthy
```

Kiểm registry mà không cần đăng nhập

```
curl.exe -s "https://ghcr.io/token?service=ghcr.io
&scope=repository:chu/anh:pull"

curl.exe -s "https://auth.docker.io/token?service=registry.docker.io
&scope=repository:library/ten-anh:pull"
```

<i>Hết.</i>